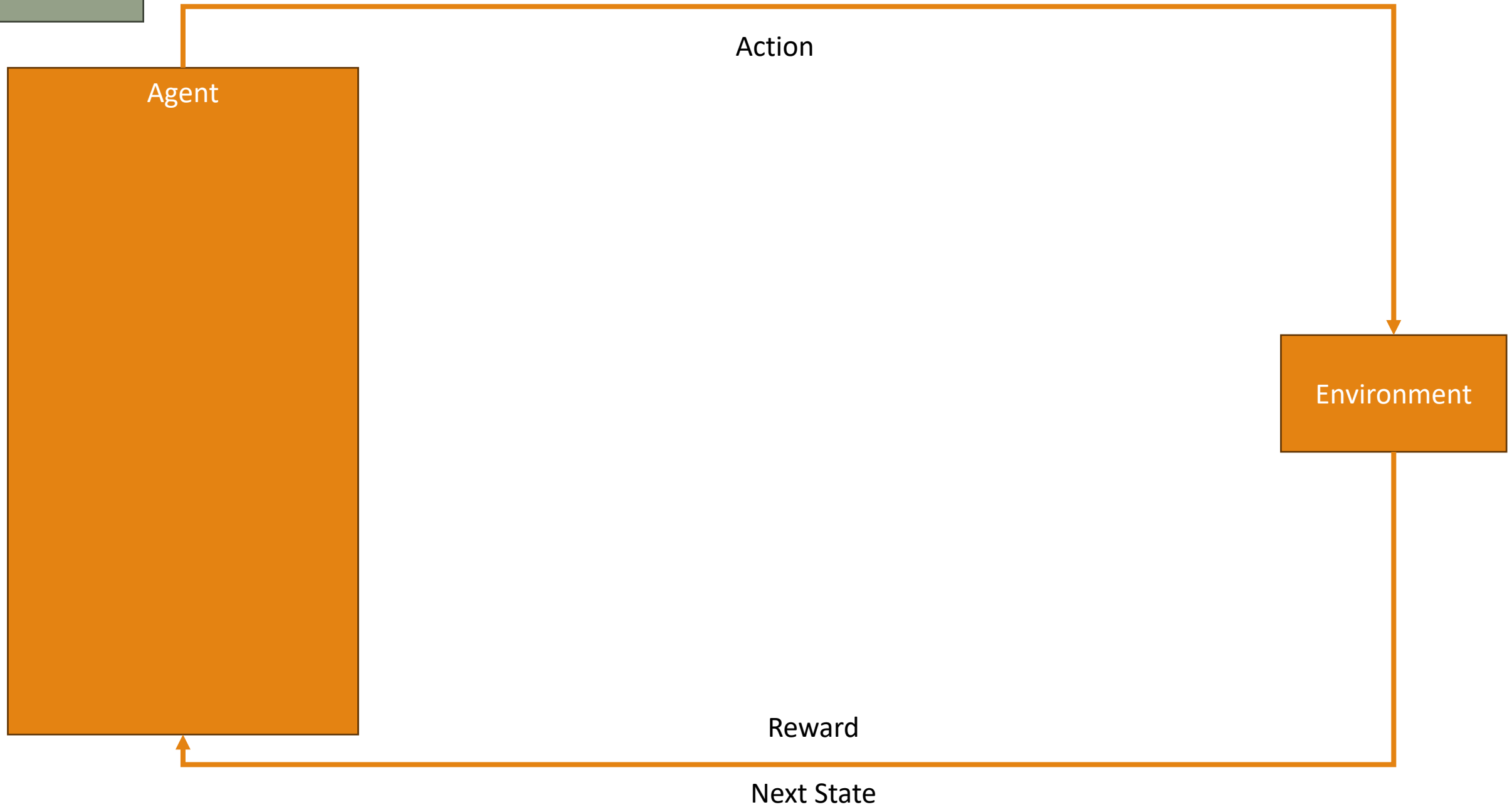
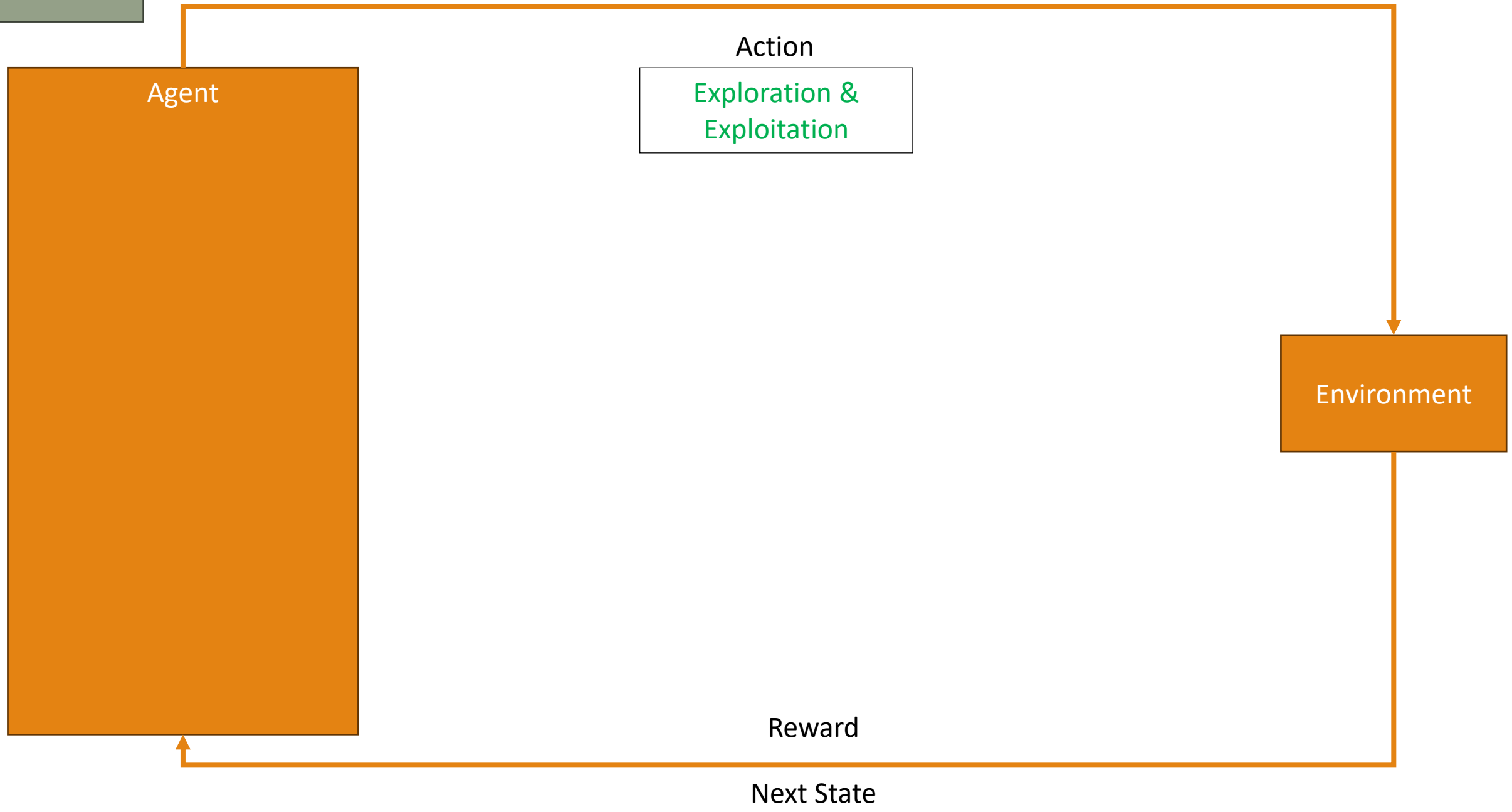
Policy-Gradient and Actor-Critic

Blink Sakulkueakulsuk

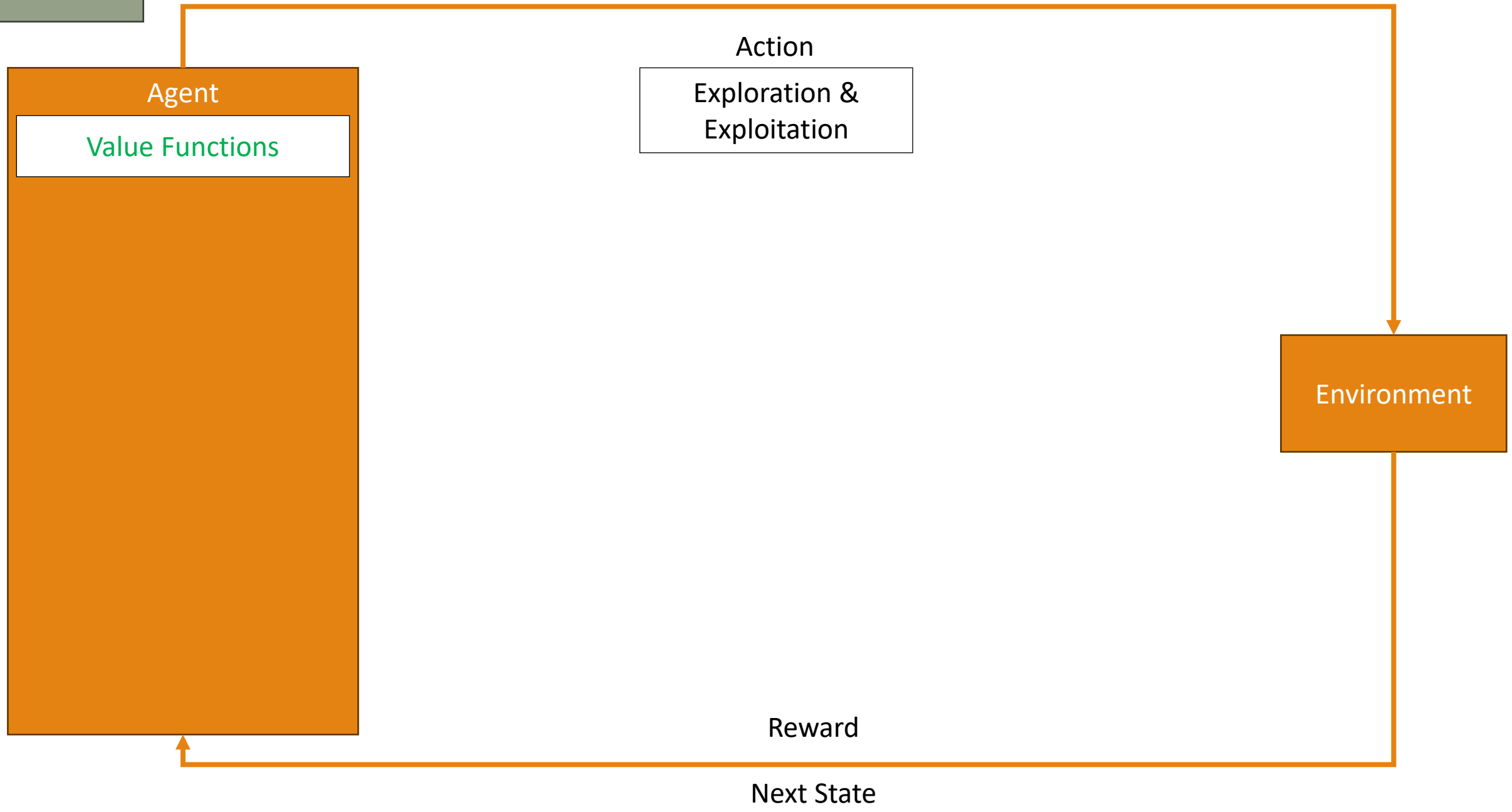
Lecture 1



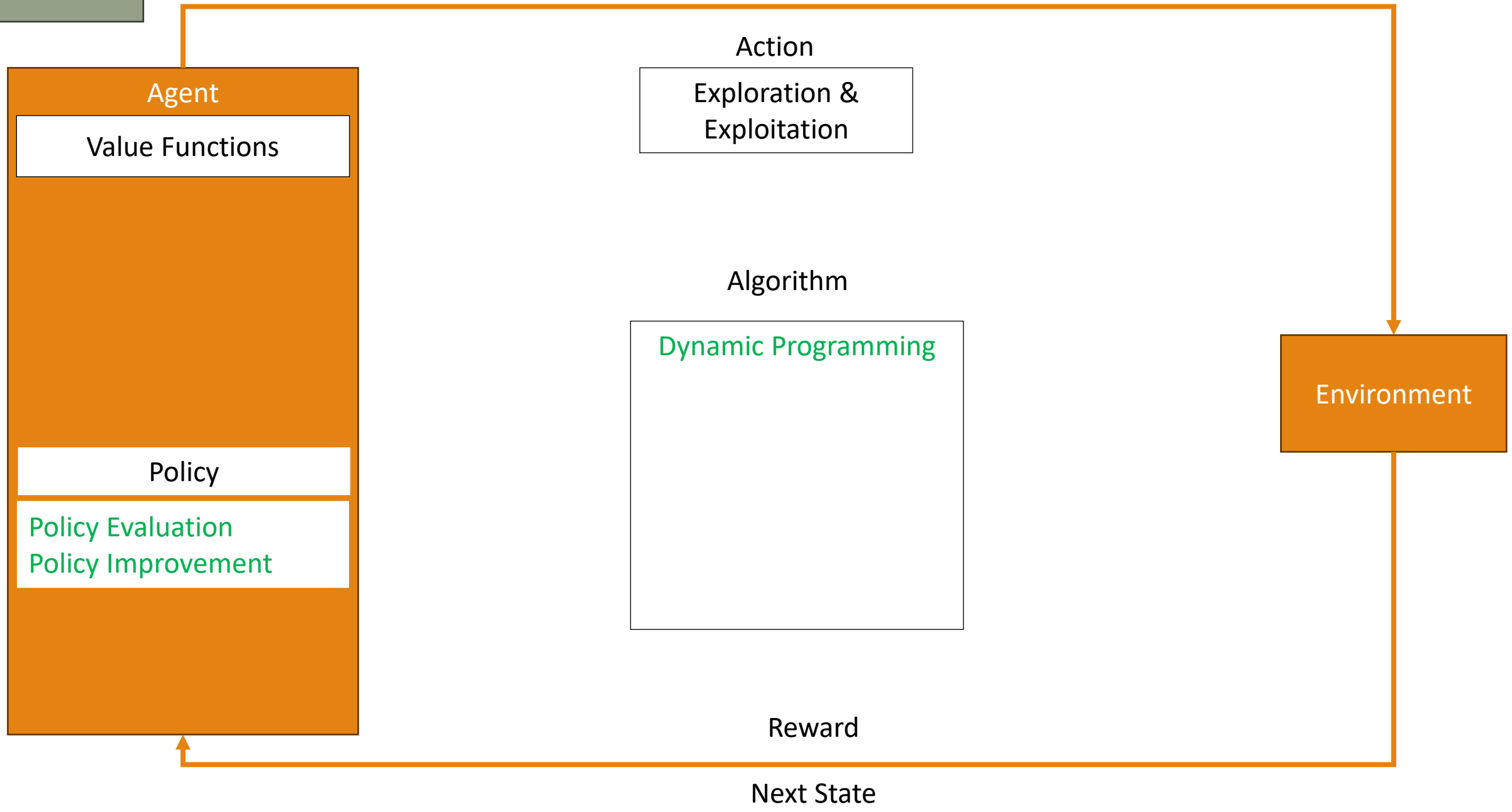
Lecture 2


Action
Exploration &
Exploitation
Agent
Environment
Reward
Next State

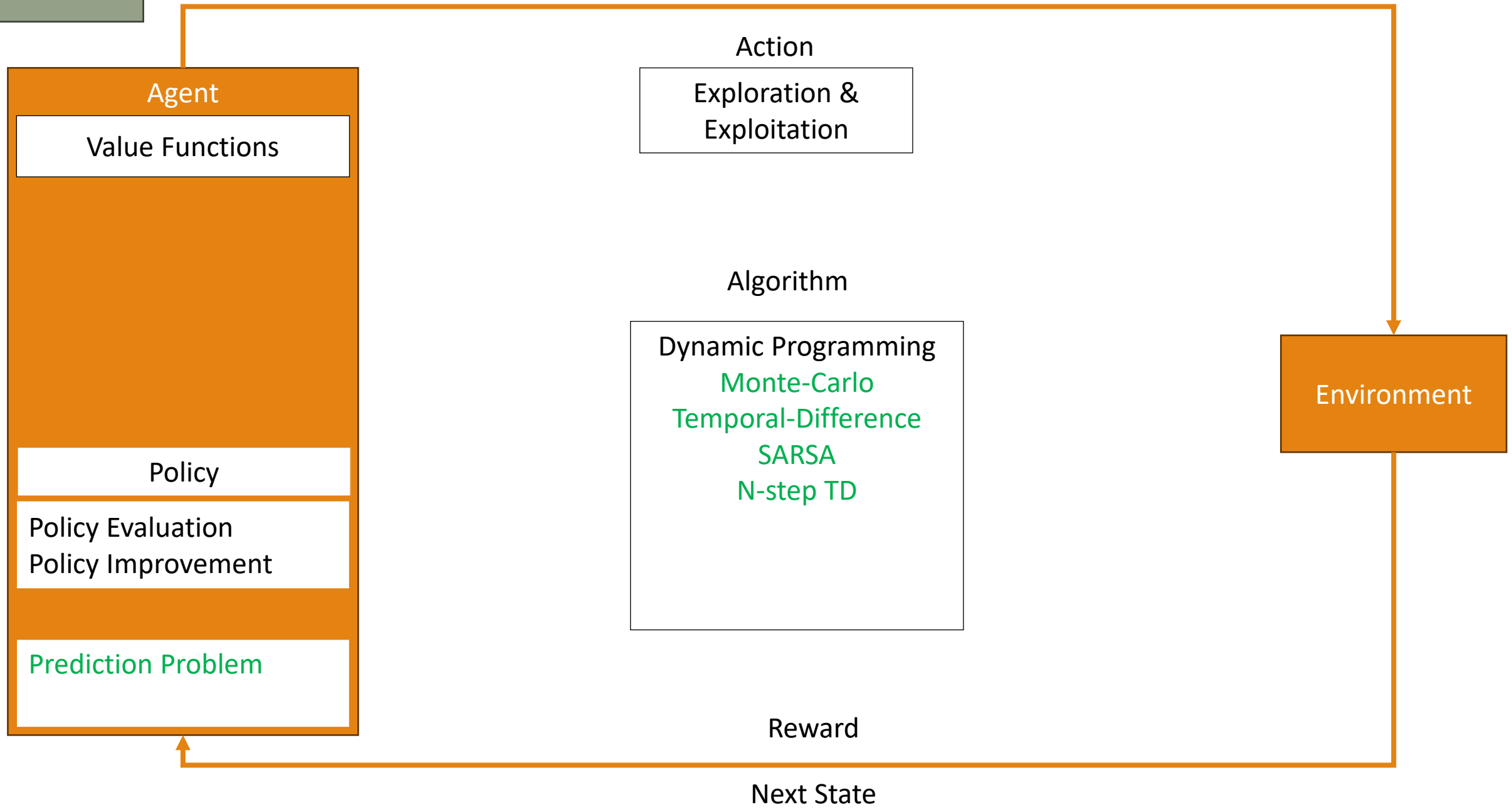
Lecture 3


Action
Exploration &
Exploitation
Agent
Value Functions
Environment
Reward
Next State

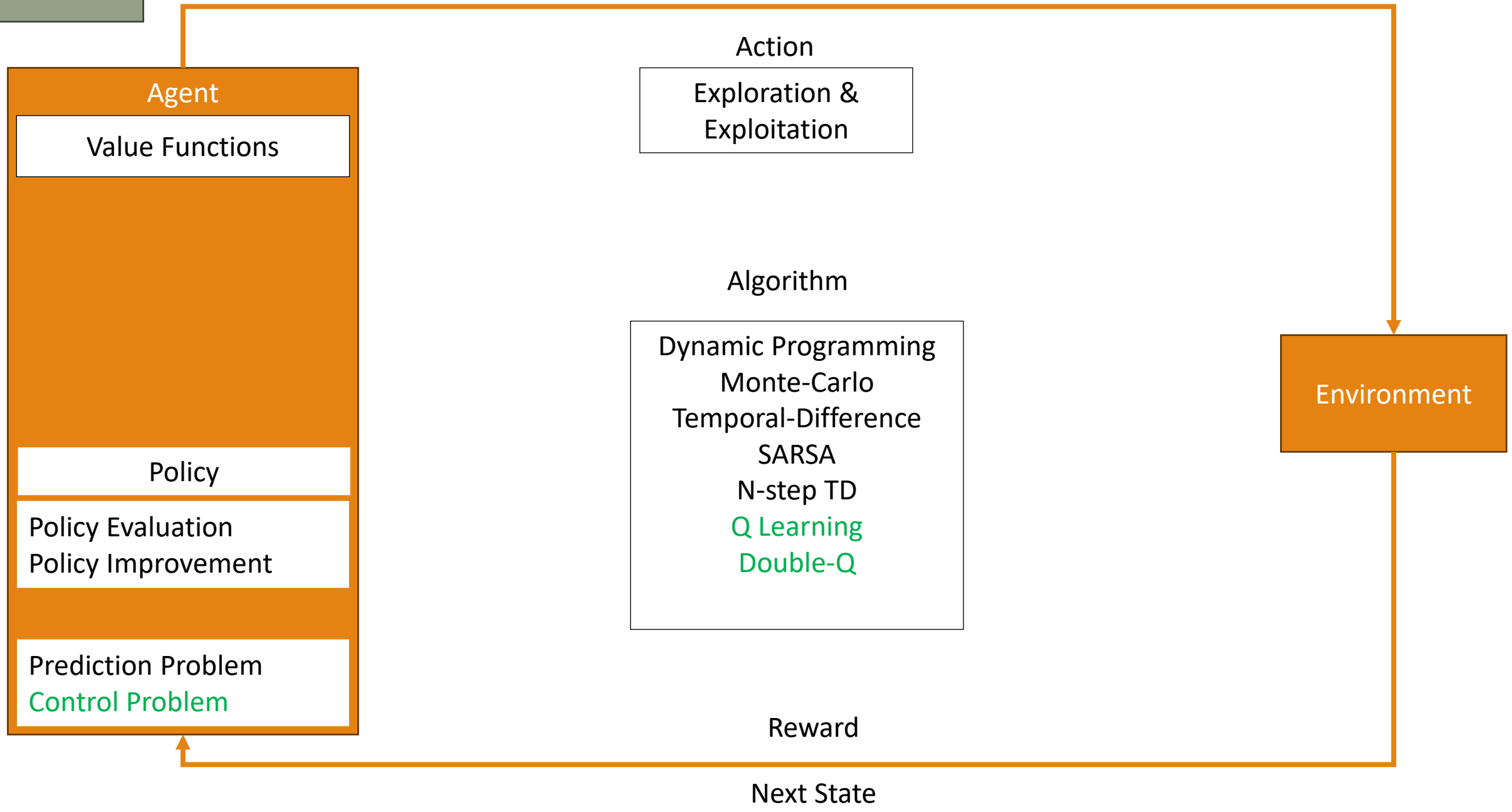
Lecture 4


Action
Exploration &
Exploitation
Agent
Value Functions
Algorithm
Dynamic Programming
Environment
Policy
Policy Evaluation
Policy Improvement
Reward
Next State

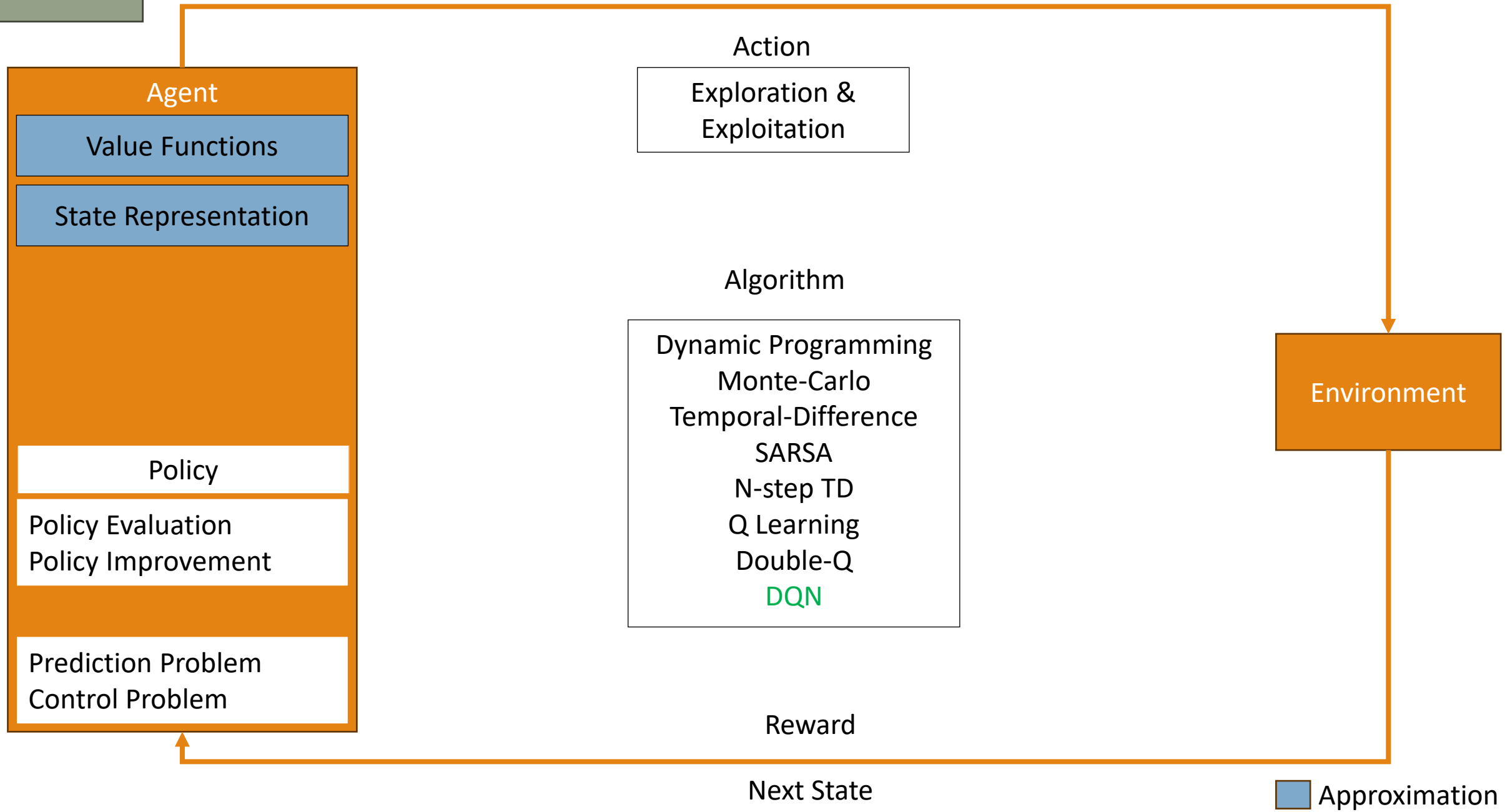
Lecture 5

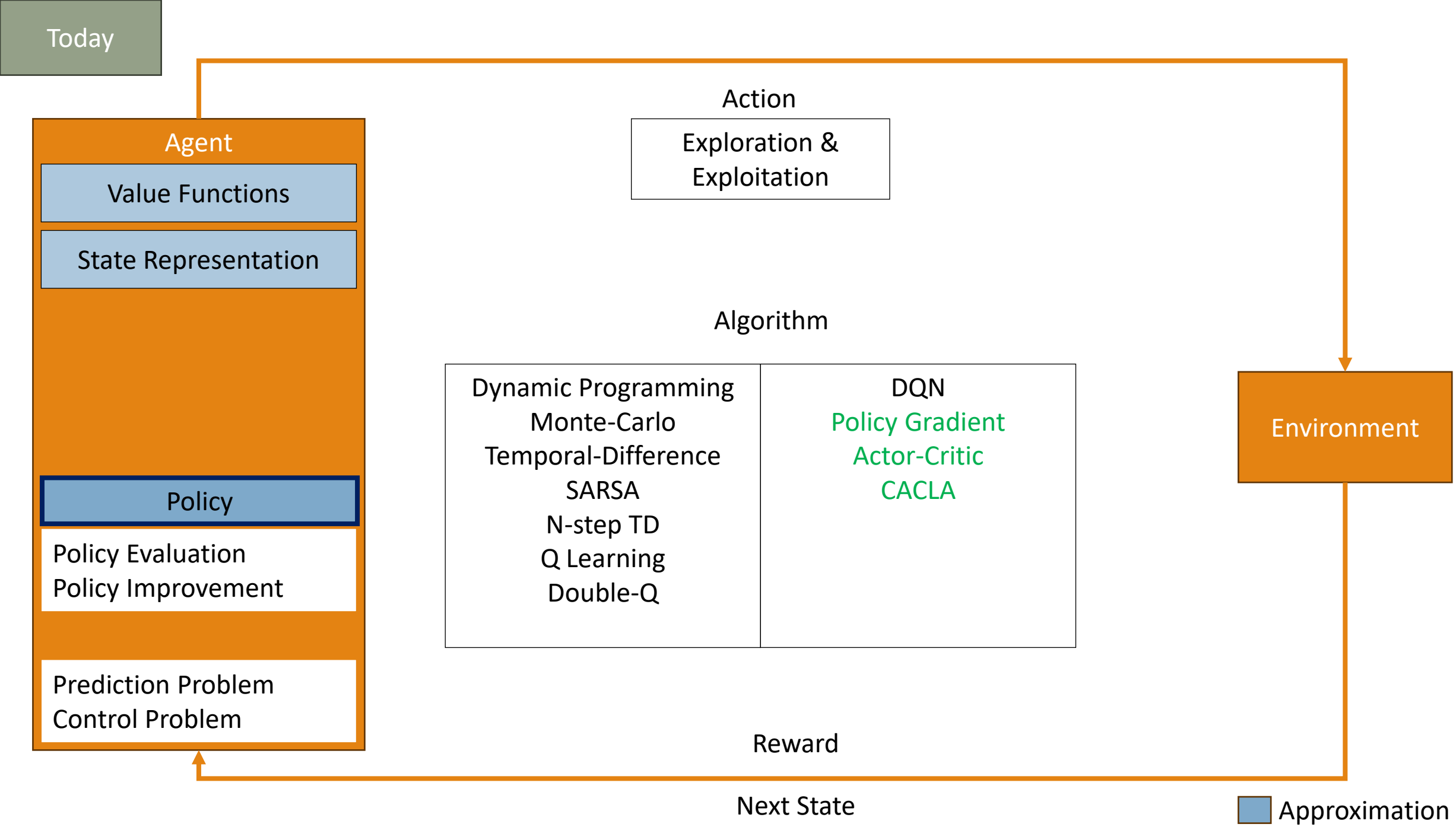

Action
Exploration &
Exploitation
Agent
Value Functions
Algorithm
Dynamic Programming
Monte-Carlo
Temporal-Difference
SARSA
N-step TD
Environment
Policy
Policy Evaluation
Policy Improvement
Prediction Problem
Reward
Next State

Lecture 6


Agent
Value Functions
Policy
Policy Evaluation
Policy Improvement
Prediction Problem
Control Problem
Action
Exploration &
Exploitation
Algorithm
Dynamic Programming
Monte-Carlo
Temporal-Difference
SARSA
N-step TD
Q Learning
Double-Q
Environment
Reward
Next State

Lecture 7


Agent
Value Functions
State Representation
Policy
Policy Evaluation
Policy Improvement
Prediction Problem
Control Problem
Action
Exploration &
Exploitation
Algorithm
Dynamic Programming
Monte-Carlo
Temporal-Difference
SARSA
N-step TD
Q Learning
Double-Q
DQN
Environment
Reward
Next State
Approximation


Today
Agent
Value Functions
State Representation
Policy
Policy Evaluation
Policy Improvement
Prediction Problem
Control Problem
Action
Exploration &
Exploitation
Algorithm
Dynamic Programming
Monte-Carlo
Temporal-Difference
SARSA
N-step TD
Q Learning
Double-Q
DQN
Policy Gradient
Actor-Critic
CACLA
Environment
Reward
Next State
Approximation

General “Mode” of Reinforcement Learning

Previously on DRL: **Value-based RL**

- Generate policy from values (e.g., ϵ-greedy on $q(s, a)$)
- Learn and approach true objectives
- Fairly well-understood

Next on DRL: **Model-based RL**

- Utilizing supervised learning to learn a model
- May learn a lot of irrelevant information from the data

General “Mode” of Reinforcement Learning

Today: **Policy-based RL**

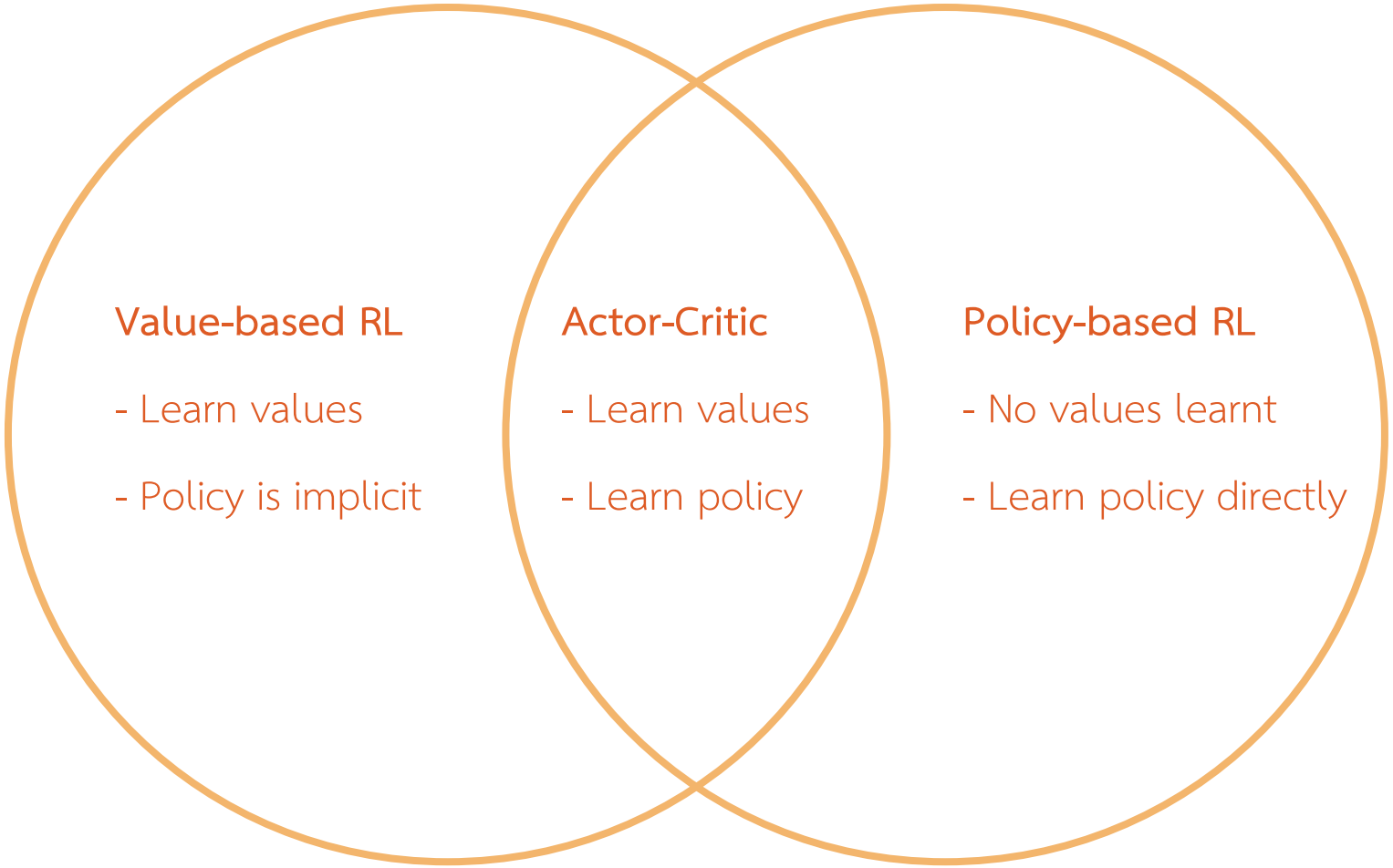
“If the agent wants to take the best action,

why don’t we learn to take the best action directly?”

This lecture will focus on learning a function approximation of the policy

$$\pi_{\boldsymbol{\theta}}(a|s) = p(a|s, \boldsymbol{\theta})$$

Value-based and Policy-based RL



Pros and Cons of Policy-based RL

Pros

- True objective
- Easy to extend to more complex problem (e.g., continuous space, high-dimensional space)
- Can learn **stochastic policy**
- May find a simple action among complex values and models

Cons

- Can get stuck in local optima
- Cannot generalize well
- Focus on a task-at-hand. Do not extract all useful information from the data

Stochastic Policy

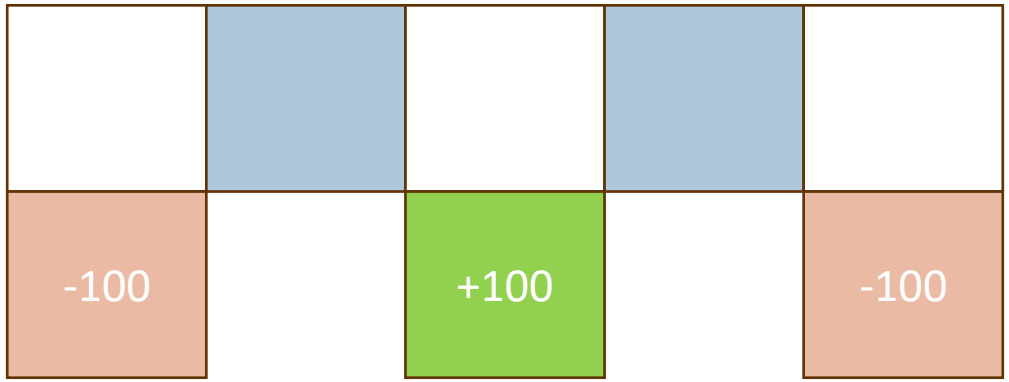
In MDPs, there is always an optimal deterministic policy

However, most problem are not fully observable

- Optimal policy may be a stochastic one to get around uncertainty of the problem

We can use gradients to search smoothly for a stochastic policy

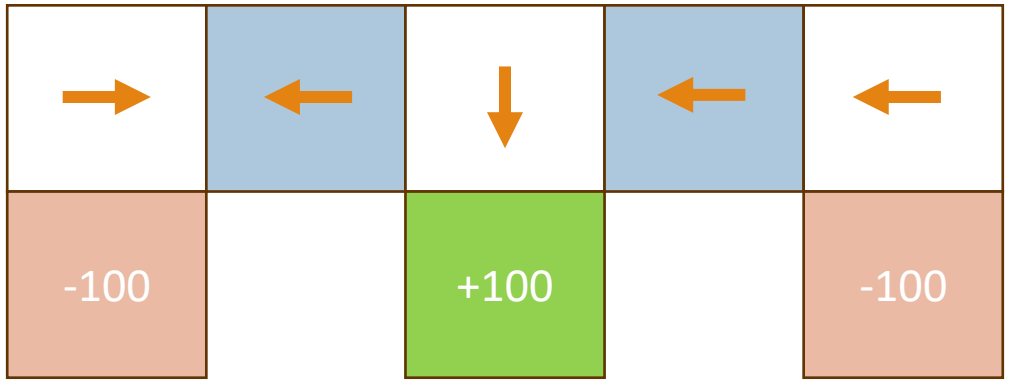
Example: Aliased Grid World



Given 4 walls (North, East, South, West) as state representation, both blue states are

$$\phi(s) = [1\ 0\ 1\ 0]$$

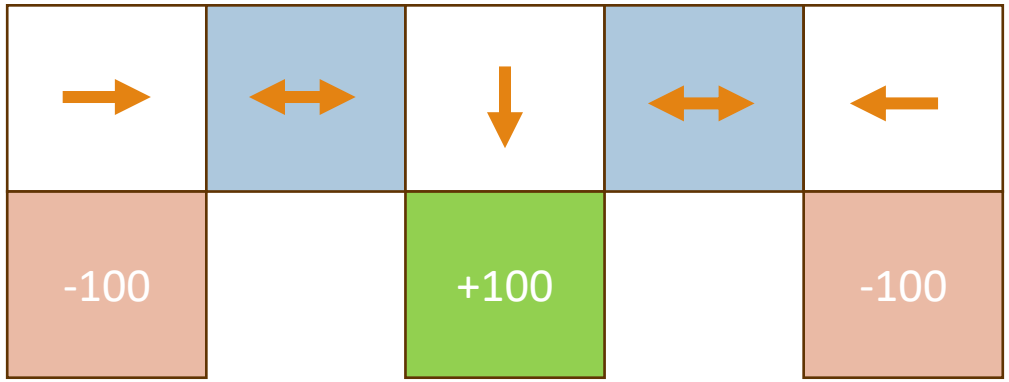
Example: Aliased Grid World



If we try to find an optimal **deterministic** policy on this map, the policy may look like the image above.

This policy will make the agent get stuck on two upper left states

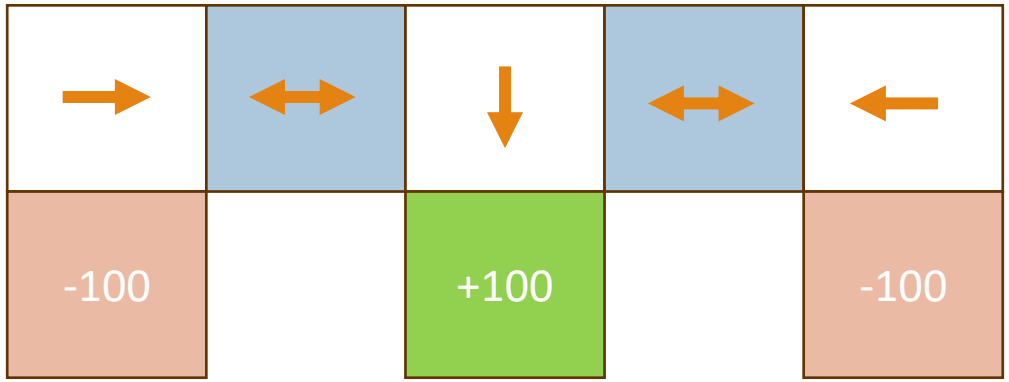
Example: Aliased Grid World



An optimal **stochastic** policy can relax the condition on both blue states, resulted in an agent reaching the goal state

$$\pi_{\boldsymbol{\theta}}(right|[1\ 0\ 1\ 0]) = 0.5$$
$$\pi_{\boldsymbol{\theta}}(left|[1\ 0\ 1\ 0]) = 0.5$$

Example: Aliased Grid World



An optimal **stochastic** policy

- Can make an agent reach the goal state
- Directly learn policy parameters
- Can learn optimal policy with non-equal probability, e.g., non-uniform distribution

Policy Objective Functions

Goal: Given a **parametric policy** $\pi_{\boldsymbol{\theta}}(s, a)$, find the **best parameters** $\boldsymbol{\theta}$

How do we know if a policy is good?

- In episodic environments, we find the **average total return per episode**
- In continuous environments, we find the **average reward per step**

Generally, a policy that gives a higher reward is better

Policy Objective Functions

Episodic-return objective

$$J_G(\boldsymbol{\theta}) = \mathbb{E}_{S_0 \sim d_0, \pi_{\boldsymbol{\theta}}}\left[\sum_t^{\infty} \gamma^t R_{t+1}\right]$$

$$= \mathbb{E}_{S_0 \sim d_0, \pi_{\boldsymbol{\theta}}}[G_0]$$

$$= \mathbb{E}_{S_0 \sim d_0}\left[\mathbb{E}_{\pi_{\boldsymbol{\theta}}}[G_t | S_t = S_0]\right]$$

$$= \mathbb{E}_{S_0 \sim d_0}\left[v_{\pi_{\boldsymbol{\theta}}}(S_0)\right]$$

d_0 is the start-state distribution

Policy Objective Functions

Average-reward objective

$$J_R(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[R_{t+1}]$$

$$= \mathbb{E}_{S_t \sim d_{\pi_{\boldsymbol{\theta}}}}\left[\mathbb{E}_{A_t \sim \pi_{\boldsymbol{\theta}}(S_t)}[R_{t+1}|S_t]\right]$$

$$= \sum_s d_{\pi_{\boldsymbol{\theta}}}(s) \sum_a \pi_{\boldsymbol{\theta}}(s,a) \sum_r p(r|s,a) r$$

$d_\pi(s) = p(S_t = s|\pi)$ is the probability of an agent being in state s in the long run. Think of this as if an agent learns for a very long time, what is the ratio of being in state s compared to other state under a policy π?

Policy Optimization

Policy-based RL tries to find the optimal policy, thus it is an optimization problem

- Find $\boldsymbol{\theta}$ that maximizes $J(\boldsymbol{\theta})$

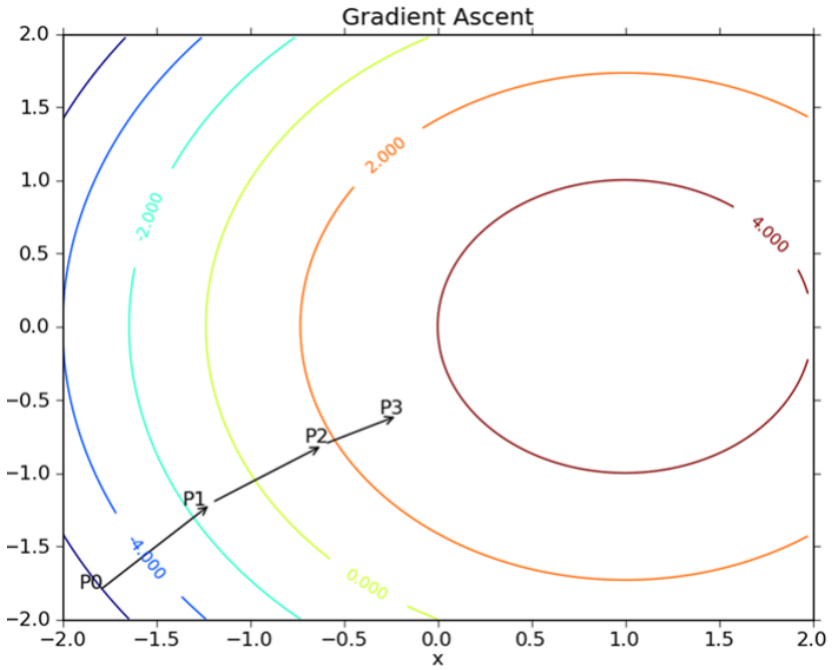
We will use **stochastic gradient ascent** to maximize the policy gradient

- This approach will utilize deep neural networks

Policy Gradient

Given an objective function $J(\boldsymbol{\theta})$, the gradient of $J(\boldsymbol{\theta})$ is

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_1} \\ \vdots \\ \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_n} \end{pmatrix}$$



https://analyticsindiamag.com/wp-content/uploads/2022/06/image-142.png

We then ascend the gradient by

$$\Delta\boldsymbol{\theta} = \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

α is a step-size parameter

Policy Gradient

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_1} \\ \vdots \\ \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_n} \end{pmatrix}$$

What exactly is this gradient?

- Recall the average reward objective $J(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[R]$

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \nabla \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[R]$$

How do we relate the reward R with the parameters $\boldsymbol{\theta}$?

Policy Gradient on Bandit Problem

Let's consider a bandit problem where $J(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[R(S, A)]$

- The reward is the expectation over d (state distribution) and π (action distribution)

- Since we are using bandit problem, assume that d doesn't depend on π for now.

To find the policy gradient, we need to find the gradient of expectations

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \nabla \, \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[R]$$

However, we cannot do it directly, since R is a scalar

- In other words, we cannot sample the episode then take the gradient afterward

- We need to rearrange this function

Policy Gradient on Bandit Problem

Let $r_{sa} = \mathbb{E}[R(S,A)|S=s, A=a]$

$$\nabla_{\boldsymbol{\theta}} \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[R(S,A)] = \nabla_{\boldsymbol{\theta}} \sum_s d(s) \sum_a \pi_{\boldsymbol{\theta}}(a|s) r_{sa}$$

$$= \sum_s d(s) \sum_a r_{sa} \nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}}(a|s)$$

$$= \sum_s d(s) \sum_a r_{sa} \pi_{\boldsymbol{\theta}}(a|s) \frac{\nabla_{\boldsymbol{\theta}}(\pi_{\boldsymbol{\theta}}(a|s)}{\pi_{\boldsymbol{\theta}}(a|s)}$$

$$= \sum_s d(s) \sum_a \pi_{\boldsymbol{\theta}}(a|s) r_{sa} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a|s)$$

$$= \mathbb{E}_{d,\pi_{\boldsymbol{\theta}}}[R(S,A) \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a|s)]$$

Policy Gradient on Bandit Problem

$$\nabla_{\boldsymbol{\theta}} \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[R(S,A)] = \mathbb{E}_{d,\pi_{\boldsymbol{\theta}}}[R(S,A)\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a|s)]$$

This equation turns **the gradient of expectation** on the left, to the **expectation of reward times the gradient of a function.**

Now, we can sample then calculate the update using

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha R_{t+1} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}_t}(A_t|S_t)$$

This function is an unbiased stochastic gradient ascend algorithm

This algorithm is known as **REINFORCE**

Reducing Variance on Policy Gradient

Let's introduce **a baseline property**. Given a value b that does not depend on the action

$$\mathbb{E}[b\nabla_{\boldsymbol{\theta}} \log \pi(A_t|S_t)] = \mathbb{E}\left[\sum_a \pi(a|S_t)\, b\nabla_{\boldsymbol{\theta}} \log \pi(A_t|S_t)\right]$$

$$= \mathbb{E}\left[\sum_a \pi(a|S_t) b \frac{\nabla_{\boldsymbol{\theta}} \pi(a|S_t)}{\pi(a|S_t)}\right]$$

$$= \mathbb{E}\left[b\nabla_{\boldsymbol{\theta}} \sum_a \pi(a|S_t)\right] = \mathbb{E}[b\nabla_{\boldsymbol{\theta}} 1] = 0$$

This implies that, we can subtract any baseline value from the update equation to reduce variance

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha(R_{t+1} - b(S_t))\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}_t}(A_t|S_t)$$

Softmax Policy Gradient

Let's use softmax function as a policy gradient.

Given some function $h(s, a)$ that output action preferences, e.g., a neural network, the softmax policy function is

$$\pi_{\boldsymbol{\theta}}(a|s) = \frac{e^{h_{\boldsymbol{\theta}}(s,a)}}{\sum_b e^{h_{\boldsymbol{\theta}}(s,b)}}$$

Softmax Policy Gradient

$$\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a|s) = \nabla_{\boldsymbol{\theta}} \log \frac{e^{h_{\boldsymbol{\theta}}(s,a)}}{\sum_b e^{h_{\boldsymbol{\theta}}(s,b)}}$$

$$= \nabla_{\boldsymbol{\theta}} \log e^{h_{\boldsymbol{\theta}}(s,a)} - \nabla_{\boldsymbol{\theta}} \log \sum_b e^{h_{\boldsymbol{\theta}}(s,b)}$$

$$= \nabla_{\boldsymbol{\theta}} h_{\boldsymbol{\theta}}(s,a) - \frac{\nabla_{\boldsymbol{\theta}} \sum_b e^{h_{\boldsymbol{\theta}}(s,b)}}{\sum_b e^{h_{\boldsymbol{\theta}}(s,b)}}$$

$$= \nabla_{\boldsymbol{\theta}} h_{\boldsymbol{\theta}}(s,a) - \frac{\sum_b e^{h_{\boldsymbol{\theta}}(s,b)} \nabla_{\boldsymbol{\theta}} h_{\boldsymbol{\theta}}(s,b)}{\sum_b e^{h_{\boldsymbol{\theta}}(s,b)}}$$

$$= \nabla_{\boldsymbol{\theta}} h_{\boldsymbol{\theta}}(s,a) - \sum_b \pi_{\boldsymbol{\theta}}(b|s) \nabla_{\boldsymbol{\theta}} h_{\boldsymbol{\theta}}(s,b)$$

Policy Gradient Theorem

To apply policy gradient to various algorithms, we can replace reward R with other values such as a return G_t or value $q_\pi(s, a)$

There are two theorems for policy gradient (Sutton et al., 2000)

- Average return per episode theorem
- Average reward per step theorem

Episodic Policy Gradient Theorem

Theorem

For any differentiable policy $\pi_{\boldsymbol{\theta}}(s, a)$, let d_0 be the starting distribution over states in which we begin an episode. Then, the policy gradient of $J(\boldsymbol{\theta}) = \mathbb{E}\left[G_0 \mid S_0 \sim d_0\right]$ is

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}\left[\sum_{t=0}^{T} \gamma^t q_{\pi_{\boldsymbol{\theta}}}(S_t, A_t) \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(A_t|S_t) \mid S_0 \sim d_0\right]$$

where

$$\begin{aligned} q_\pi(s, a) &= \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \end{aligned}$$

Episodic Policy Gradient Algorithm

$$\nabla_{\boldsymbol{\theta}} J_{\boldsymbol{\theta}}(\pi) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{T} \gamma^t q_{\pi}(S_t, A_t) \nabla_{\boldsymbol{\theta}} \log \pi(A_t|S_t)\right]$$

Typically, we calculate the sum first, then update the gradient, e.g.,

$$\Delta \boldsymbol{\theta}_t = \gamma^t G_t \nabla_{\boldsymbol{\theta}} \log \pi(A_t|S_t)$$

Then

$$\nabla_{\boldsymbol{\theta}} J_{\boldsymbol{\theta}}(\pi) = \mathbb{E}_{\pi}\left[\sum_{t} \Delta \boldsymbol{\theta}_t\right]$$

Average Reward Policy Gradient Theorem

Theorem

For any differentiable policy $\pi_{\boldsymbol{\theta}}(s, a)$, the policy gradient of $J(\boldsymbol{\theta}) = \mathbb{E}[R \mid \pi]$ is

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\pi}[q_{\pi_{\boldsymbol{\theta}}}(S_t, A_t) \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(A_t | S_t)]$$

where

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[R_{t+1} - \rho + q_{\pi}(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

$$\rho = \mathbb{E}_{\pi}[R_{t+1}] \qquad \textit{(Note: global average, not conditioned on state or action)}$$

(Expectation is over both states and actions)

Average Reward Policy Gradient Theorem

Alternatively (but equivalently):

Theorem

For any differentiable policy $\pi_{\boldsymbol{\theta}}(s, a)$, the policy gradient of $J(\boldsymbol{\theta}) = \mathbb{E}[R \mid \pi]$ is

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\pi}[R_{t+1} \sum_{n=0}^{\infty} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(A_{t-n} | S_{t-n})]$$

(Expectation is over both states and actions)

Policy Gradients Variance Reduction

Recall that $\mathbb{E}[b(S_t)\nabla_{\boldsymbol{\theta}} \log \pi(A_t|S_t)] = 0$ for all $b(S_t)$ that does not depend on A_t

So we can add a common baseline $v_\pi(S_t)$ into the policy gradient equation

$$\nabla_{\boldsymbol{\theta}} J_{\boldsymbol{\theta}}(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t (q_\pi(S_t, A_t) - v_\pi(S_t)) \nabla_{\boldsymbol{\theta}} \log \pi(A_t|S_t) \right]$$

Policy Gradients Variance Reduction

$$\nabla_{\boldsymbol{\theta}} J_{\boldsymbol{\theta}}(\pi) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{T} \gamma^t (q_{\pi}(S_t, A_t) - v_{\pi}(S_t)) \nabla_{\boldsymbol{\theta}} \log \pi(A_t|S_t)\right]$$

Since we have a value function here, it also means that we can approximate this function! So we have two approximations here.

$$v_{\mathbf{w}}(s) \approx v_{\pi}(s)$$
$$q_{\boldsymbol{\pi}}(S_t, A_t) \approx G_t$$

Critics

$$v_{\mathbf{w}}(s) \approx v_{\pi}(s)$$

A critic is a value function, learnt from policy evaluation.

- Basically, it evaluates the value $v_{\pi_{\boldsymbol{\theta}}}$ given a policy $\pi_{\boldsymbol{\theta}}$ for the current parameters $\boldsymbol{\theta}$

We can use all function approximation technique in the past lectures to solve

- Monte-Carlo policy evaluation
- Temporal-Difference learning
- n-step TD

Actor-Critic

Critic Update parameters $\boldsymbol{w}$ of $v_{\boldsymbol{w}}$ by TD (e.g., one-step) or MC

Actor Update $\boldsymbol{\theta}$ by policy gradient

function ONE-STEP ACTOR CRITIC

Initialise s, $\boldsymbol{\theta}$

for t = 0, 1, 2, ... **do**

Sample $A_t \sim \pi_{\boldsymbol{\theta}}(S_t)$

Sample R_{t+1} and S_{t+1}

$\delta_t = R_{t+1} + \gamma v_{\boldsymbol{w}}(S_{t+1}) - v_{\boldsymbol{w}}(S_t)$ [one-step TD-error, or **advantage**]

$\boldsymbol{w} \leftarrow \boldsymbol{w} + \beta\, \delta_t\, \nabla_{\boldsymbol{w}} v_{\boldsymbol{w}}(S_t)$ [TD(0)]

$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, \delta_t\, \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(A_t \mid S_t)$ [Policy gradient update (ignoring γ^t term)]

Continuous Actions

Actions we used so far are discrete. What if we want actions with continuous values? E.g., torque of a motor, joint position,...

- How to approximate $q(s, a)$
- How to compute $\max_a q(s, a)$

We can modify algorithms discussed today to use in continuous space

Gaussian Policy

Let's look at gaussian policy

- An action is derived from a policy with normal distribution $A_t \sim N(\mu_{\boldsymbol{\theta}}(S_t), \sigma^2)$

- $\mu_{\boldsymbol{\theta}}(s)$ denotes a mean in the form of a function of state

The gradient of the log of the policy is

$$\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a|s) = \frac{A_t - \mu_{\boldsymbol{\theta}}(S_t)}{\sigma^2} \nabla \mu_{\boldsymbol{\theta}}(s)$$

Policy Gradient with Gaussian Policy

We then plug in the gradient into the update

$$\begin{aligned}\boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t + \alpha(G_t - v(S_t))\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}_t}(A_t|S_t) \\ &= \boldsymbol{\theta}_t + \alpha(G_t - v(S_t))\frac{A_t - \mu_{\boldsymbol{\theta}}(S_t)}{\sigma^2}\nabla\mu_{\boldsymbol{\theta}}(s)\end{aligned}$$

If an action A_t is better (give more return) than the approximation from the policy ($\mu_{\boldsymbol{\theta}}(S_t)$), then adjust the approximation toward the action

Continuous Actor-Critic Learning Automaton (CACLA)

1. $a_t = \text{Actor}_{\boldsymbol{\theta}}(S_t)$ (get current (continuous) action proposal)
2. $A_t \sim \pi(\cdot|S_t, a_t)$ (e.g., $A_t \sim \mathcal{N}(a_t, \Sigma)$) (explore)
3. $\delta_t = R_{t+1} + \gamma v_{\boldsymbol{w}}(S_{t+1}) - v_{\boldsymbol{w}}(S_t)$ (compute TD error)
4. Update $v_{\boldsymbol{w}}(S_t)$ (e.g., using TD) (policy evaluation)
5. If $\delta_t > 0$, update $\text{Actor}_{\boldsymbol{\theta}}(S_t)$ towards A_t (policy improvement)

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t + \beta(A_t - a_t)\nabla_{\boldsymbol{\theta}_t}\text{Actor}_{\boldsymbol{\theta}_t}(S_t)$$

6. If $\delta_t \leq 0$, do not update $\text{Actor}_{\boldsymbol{\theta}}$